

In [1]:

```
#Import relevant modules
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import math
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

In [2]:

```
#Extract data
data = 'D:\\AI Course\\Data Engineering\\stroke_data.csv'
```

In [3]:

```
#Read the data into the workbook
Data = pd.read_csv(data)
```

In [4]:

```
#Create the dataframe that we will be using
Stroke_Data = pd.DataFrame(Data)
Stroke_Data
```

Out[4]:

	sex	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	1.0	63	0	1	1	4	1	228.69	36.6	1	1
1	1.0	42	0	1	1	4	0	105.92	32.5	0	1
2	0.0	61	0	0	1	4	1	171.23	34.4	1	1
3	1.0	41	1	0	1	3	0	174.12	24.0	0	1
4	1.0	85	0	0	1	4	1	186.21	29.0	1	1
...	...	...	...	...	...	...	...	...	...	...	...
40905	1.0	38	0	0	0	4	1	120.94	29.7	1	0
40906	0.0	53	0	0	1	4	0	77.66	40.8	0	0
40907	1.0	32	0	0	1	2	0	231.95	33.2	0	0
40908	1.0	42	0	0	1	3	0	216.38	34.5	0	0
40909	1.0	35	0	0	0	4	0	95.01	28.0	0	0

40910 rows × 11 columns

In [5]:

```
#Undertsand if there are any gaps in the data
Stroke_Data.isnull().sum()
```

Out[5]:

```
sex          3
age          0
hypertension  0
heart_disease 0
ever_married 0
work_type    0
Residence_type 0
avg_glucose_level 0
bmi          0
smoking_status 0
stroke       0
dtype: int64
```

In [6]:

```
Stroke_Data.dropna(inplace=True)
```

In [7]:

```
#Remove any null values
Stroke_Data.isnull().sum()
```

Out[7]:

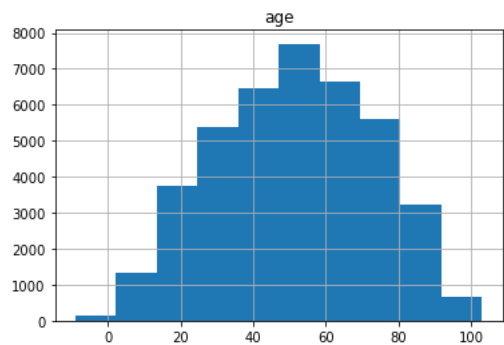
```
sex          0
age          0
hypertension 0
heart_disease 0
ever_married 0
work_type    0
Residence_type 0
avg_glucose_level 0
bmi          0
smoking_status 0
stroke       0
dtype: int64
```

In [8]:

```
#See the range of ages in the study
Stroke_Data.hist(column='age')
```

Out[8]:

array([[<AxesSubplot:title={'center':'age'}>]], dtype=object)



In [9]:

```
#Seeing how many ages are actually negtaive and how much of the data is corrupt
Stroke_Data[Stroke_Data['age']<0]
```

10278	1.0	-1	1	0	1	4	0	243.58	33.6	0	1
10423	1.0	-3	0	0	1	2	0	190.32	31.4	0	1
10458	1.0	-4	0	0	1	4	1	78.18	30.8	0	1
10489	0.0	-1	0	1	1	4	1	91.92	35.9	1	1
10525	1.0	-4	0	0	1	4	1	86.23	30.0	1	1
10549	1.0	-4	1	0	1	4	0	249.31	35.8	0	1
10616	1.0	-1	1	0	1	3	0	167.41	30.9	0	1
10768	1.0	-3	0	0	0	4	0	88.92	22.9	0	1
10826	1.0	-4	0	0	1	3	1	74.96	26.6	0	1
10834	1.0	-1	0	0	1	2	0	103.43	27.3	1	1
10871	1.0	-1	0	0	1	4	1	185.17	40.4	1	1
11217	1.0	-3	1	0	1	3	1	124.13	31.4	0	1
11713	1.0	-6	0	1	1	2	0	120.46	36.8	1	1

In [10]:

```
#Dropping all negative ages
Stroke_Data.drop(Stroke_Data[Stroke_Data['age']<0].index,inplace=True)
```

In [11]:

```
#Making sure all negative ages have been removed
Stroke_Data[Stroke_Data['age']<0]
```

Out[11]:

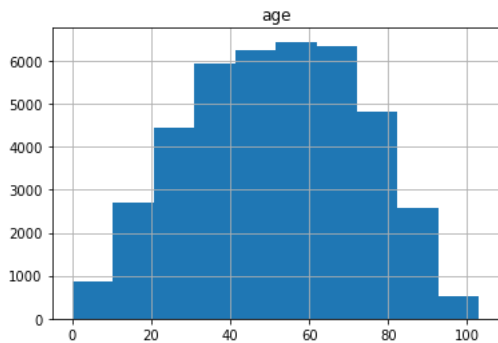
```
sex age hypertension heart_disease ever_married work_type Residence_type avg_glucose_level bmi smoking_status stroke
```

In [12]:

```
#Checking the new range of ages
Stroke_Data.hist(column='age')
```

Out[12]:

```
array([[<AxesSubplot:title={'center':'age'}>]], dtype=object)
```



In [13]:

```
#what do I want to use to find my target
feature_data = Stroke_Data[["sex", "age", "hypertension", "heart_disease", "ever_married", "work_type", "Residence_type", "bmi",
                             "avg_glucose_level", "smoking_status"]]
```

In [14]:

```
#what is my target/what am I trying to calculate
target_data = Stroke_Data[["stroke"]]
```

In [15]:

```
x = feature_data
y = target_data
```

In [16]:

```
#Split the data into test and train so we can evaluate the model later
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=4)
print(x_test.shape)
print(y_test.shape)
```

```
(8170, 10)
(8170, 1)
```

In [17]:

```
#create the Logistic regression model
logreg = LogisticRegression()
logreg.fit(x_train, np.ravel(y_train))
```

C:\Users\kieran\anaconda3\lib\site-packages\sklearn\linear_model_logistic.py:762: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html> (<https://scikit-learn.org/stable/modules/preprocessing.html>)
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression (https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression)
n_iter_i = _check_optimize_result(

Out[17]:

```
LogisticRegression()
```

In [18]:

```
#measure model's results against the test results
y_pred = logreg.predict(x_test)
```

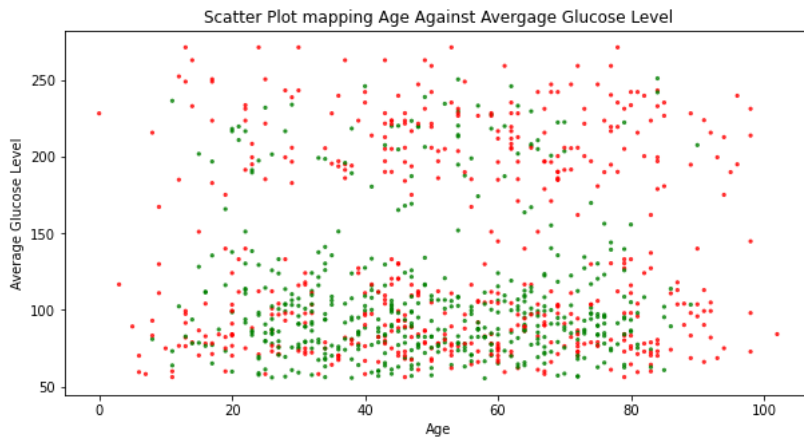
In [19]:

```
#calculate accuracy of the model
score = accuracy_score(y_test, y_pred, normalize=True)
print("The model is", score*100, "% accurate")
```

```
The model is 69.02080783353733 % accurate
```

In [20]:

```
#Showing the relationship between age and glucose Level showing individuals with strokes red=stroke, green=no stroke
Stroke_sample = Stroke_Data.sample(1000)
plt.figure(figsize=(10,5))
plt.scatter(Stroke_sample['age'],Stroke_sample['avg_glucose_level'],
            c=Stroke_sample['stroke'].map({0:'green',1:'red'}),s=5, alpha=0.75)
plt.title('Scatter Plot mapping Age Against Average Glucose Level')
plt.xlabel('Age')
plt.ylabel('Average Glucose Level')
plt.show()
```

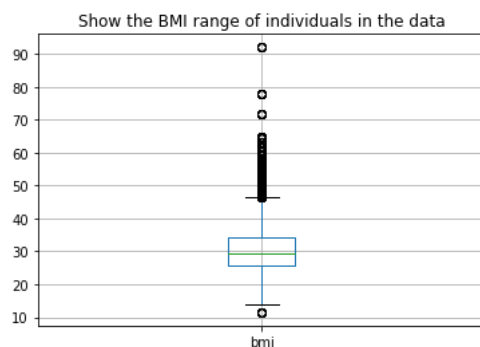


In [21]:

```
#Boxplot to show BMI ranges
Stroke_plot = Stroke_Data.boxplot(column=['bmi'])
plt.title('Show the BMI range of individuals in the data')
Stroke_plot.plot()
```

Out[21]:

[]



In [22]:

```
#Create a connection between python and SQL
from sqlalchemy import create_engine
engine = create_engine("postgresql://postgres:Kieran1998!@localhost:5432/DE_SUMM2")
```

In [23]:

```
#Upload Dataframe to SQL
Stroke_Data.to_sql('Stroke_Data',engine)
```

In []: